

國立臺灣大學
National Taiwan University

National Taiwan University

School of Computer Science and Information Engineering

NTU ADL 2025 Fall HW1

Author:
Min-Sheng,Tsai

Lecturer:
Vivian

An Assignment submitted for the UoS:

CSIE5431 Applied Deep Learning

February 18, 2026

Contents

1 Abstract **2**

2 Data processing **2**

3 Modeling with BERTs and their variants **2**

4 Curves **3**

5 Pre-trained vs Not Pre-trained(From scratch) **3**

5.1 Fine-Tuning 3

5.2 From scratch 4

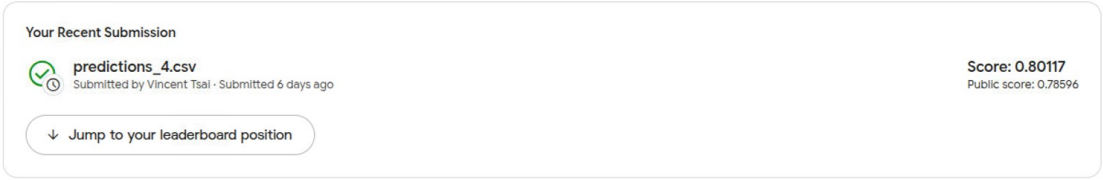


Figure 1: Higher-Strong-baseline

1 Abstract

Model Performance (11%) Kaggle simple baseline: public (2%), private (3%). Kaggle strong baseline: public (2%), private (3%). TA can reproduce your results without human intervention. (1%) 0 point if we cannot reproduce your submission after human intervention. Your reproduction results should match the baselines passed by your final Kaggle submission, or you will lose points. Report (9% + 2% Bonus)

X for each token is the sum of three embeddings:

$$
\begin{aligned}
X &= E_{\text{token}} + E_{\text{position}} + E_{\text{segment}} \\
E_{\text{token}} &= \text{Token Embedding} \\
E_{\text{position}} &= \text{Positional Embedding} \\
E_{\text{segment}} &= \text{Sentence (Segment) Embedding}
\end{aligned}
$$

2 Data processing

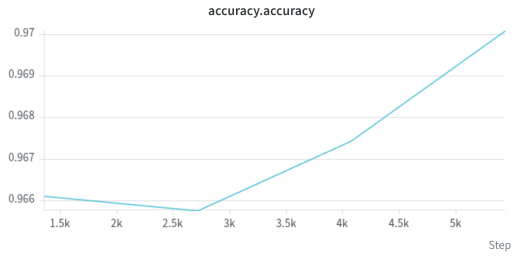
Describe in detail about the tokenization algorithm you use. You need to explain what algorithm:

I basically used the WordPiece tokenization algorithm through Hugging Face's BERT tokenizer; the tokenizer converts each question and its corresponding context into tokens and then maps those tokens. to integer IDs in a Pre-trained vocabulary. WordPiece represents text using subword units, allowing many uncommon words to be represented by combinations of known tokens.

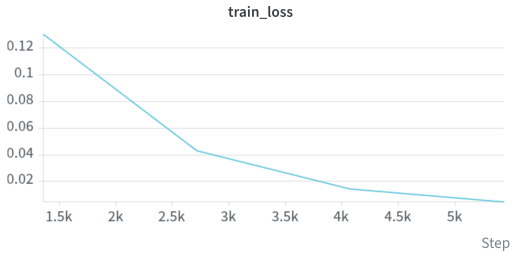
3 Modeling with BERTs and their variants

1. google-bert/bert-base-chinese
2. hfl/chinese-roberta-wwm-ext
3. hfl/chinese-pert-base
4. hfl/chinese-lert-base

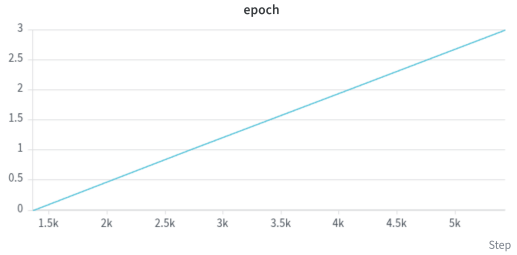
4 Curves



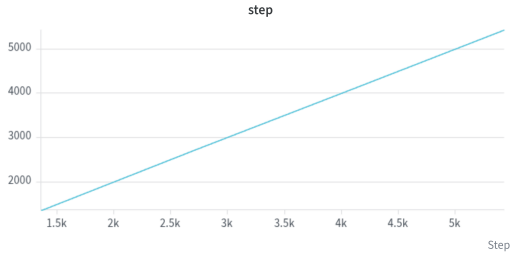
(a) Accuracy



(b) Training loss



(c) Epoch



(d) Step

Figure 2: Training metrics and progress.

5 Pre-trained vs Not Pre-trained(From scratch)

By coding

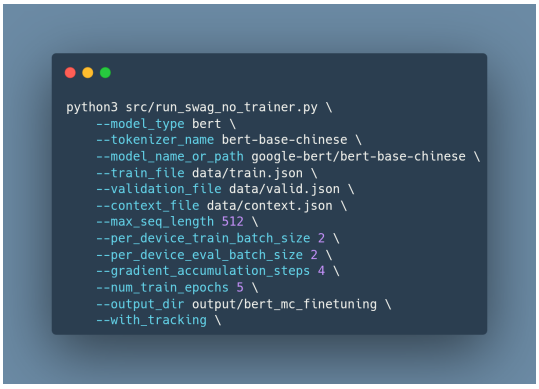
1. From scratch:
–model-type bert
–tokenizer-name bert-base-chinese

2. Fine Tuing(Adding pre-train model):
–model-name-or-path bert-base-chinese

By theorem
Basically we would called a pre-train model, then fine tuing the model, this would be the section 2 added the BERT-BASE-PRETRAIN model from the huggingface.

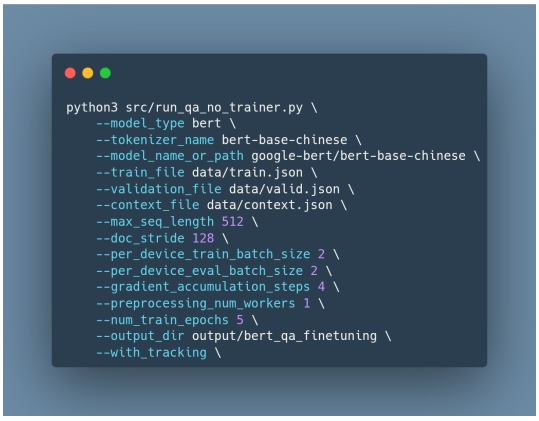
5.1 Fine-Tuning

"Val EM Score": [76.9690927218345] "Val Loss": [0.6257049296795564]



```
python3 src/run_swag_no_trainer.py \
    --model_type bert \
    --tokenizer_name bert-base-chinese \
    --model_name_or_path google-bert/bert-base-chinese \
    --train_file data/train.json \
    --validation_file data/valid.json \
    --context_file data/context.json \
    --max_seq_length 512 \
    --per_device_train_batch_size 2 \
    --per_device_eval_batch_size 2 \
    --gradient_accumulation_steps 4 \
    --num_train_epochs 5 \
    --output_dir output/bert_mc_finetuning \
    --with_tracking \
```

Figure 3: MC fine-tuning



```
python3 src/run_qa_no_trainer.py \
    --model_type bert \
    --tokenizer_name bert-base-chinese \
    --model_name_or_path google-bert/bert-base-chinese \
    --train_file data/train.json \
    --validation_file data/valid.json \
    --context_file data/context.json \
    --max_seq_length 512 \
    --doc_stride 128 \
    --per_device_train_batch_size 2 \
    --per_device_eval_batch_size 2 \
    --gradient_accumulation_steps 4 \
    --preprocessing_num_workers 1 \
    --num_train_epochs 5 \
    --output_dir output/bert_qa_finetuning \
    --with_tracking \
```

Figure 4: QA fine-tuning

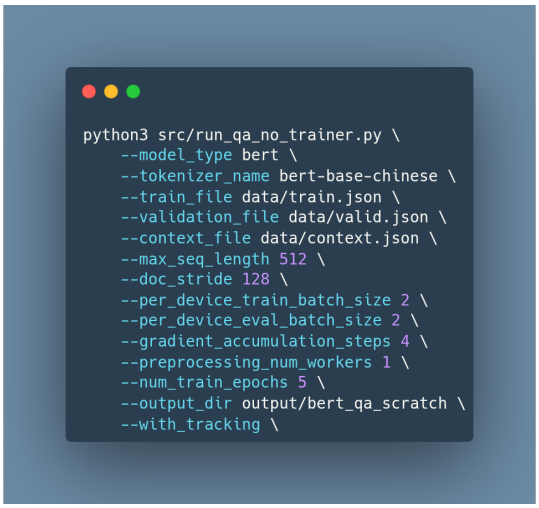
5.2 From scratch

"Val EM Score": [4.253904951811233] "Val Loss": [4.3690661540416 88]



```
python3 src/run_swag_no_trainer.py
    --model_type bert \
    --tokenizer_name bert-base-chinese \
    --train_file data/train.json \
    --validation_file data/valid.json \
    --context_file data/context.json \
    --max_seq_length 512 \
    --per_device_train_batch_size 2 \
    --per_device_eval_batch_size 2 \
    --gradient_accumulation_steps 4 \
    --num_train_epochs 5 \
    --output_dir output/bert_mc_scratch \
    --with_tracking \
    --debug \
    --dataset_name swag
```

Figure 5: MC fine-tuning



```
python3 src/run_qa_no_trainer.py \
    --model_type bert \
    --tokenizer_name bert-base-chinese \
    --train_file data/train.json \
    --validation_file data/valid.json \
    --context_file data/context.json \
    --max_seq_length 512 \
    --doc_stride 128 \
    --per_device_train_batch_size 2 \
    --per_device_eval_batch_size 2 \
    --gradient_accumulation_steps 4 \
    --preprocessing_num_workers 1 \
    --num_train_epochs 5 \
    --output_dir output/bert_qa_scratch \
    --with_tracking \
```

Figure 6: QA fine-tuning